CSCI 126 Report - Asteroid Close-Approach Database Analysis

Author - Jason Anthony Avalos

1.1) Dataset Selection

Source - https://arxiv.org/pdf/2409.02150

Google Colab - https://colab.research.google.com/drive/1eo5UWdw0pC0uSxd6UcKhP9SJvUdllirF?usp=sharing

Asteroid Dataset (https://www.kaggle.com/datasets/sakhawat18/asteroid-dataset) - The **Asteroid Dataset** is a dataset compiled by **Mir Sakhawat Hossain** who is a self-proclaimed data science, machine learning, and deep learning enthusiast. The data is collected from the Jet Propulsion Laboratory (JPL) of California Institute of Technology which is an organization managed by National Aeronautics and Space Administration (NASA). There are 45 total features available for analysis.

NeoWs Dataset (https://www.kaggle.com/datasets/alvinb/neows-hazardous-asteroid-dataset) - The **NeoWs Hazardous Asteroid Dataset** is a dataset compiled by **Christopher Alvin Buana** who is the author of the paper I am referencing and a researcher at the National Central University of Taiwan. The data is described as “The dataset describes the features of an asteroids such as the physical measurements, the date of the asteroids and the labels to determine whether the asteroid is hazardous or normal to earth." There are 41 total features available for analysis.

1.2) Attribute Decoding and Terminology

Several attributes in the original datasets use shortened scientific or database-style names, so they are clarified here before the analysis. This is important because the SQL queries rely on these terms to describe asteroid identity, orbit characteristics, physical size, velocity, and close-approach distance.

Attribute / Term	Meaning
`spkid`	A unique asteroid identifier used in the asteroid dataset. This is used as the primary key in the asteroid table.
`neo_reference_id`	The asteroid identifier used in the close-approach dataset. This connects each close-approach event to an asteroid record.
`approach_id`	A generated primary key used to uniquely identify each close-approach event.
`NEO`	Near-Earth Object. This refers to an object whose orbit brings it near Earth’s orbit.
`PHA` / `hazardous`	Potentially Hazardous Asteroid classification. In this project, hazardous status is used to compare risk-related records.
`H` / `absolute_magnitude`	A brightness measurement used in astronomy. Lower values generally indicate brighter and often larger objects.
`MOID` / `minimum_orbit_intersection_distance`	Minimum Orbit Intersection Distance. This measures how close an asteroid’s orbit comes to Earth’s orbit.
`orbit_class`	The asteroid’s orbital classification, such as APO, ATE, AMO, or IEO.

Clarifying these terms at the beginning of the report helps make the later SQL analysis easier to understand. For example, queries involving miss_dist_km, MOID, orbit_class, and hazardous require domain-specific interpretation, so defining them early prevents confusion when discussing the findings.

Orbit Class Definitions

The orbit_class attribute describes the asteroid’s orbital group. These classes are based on orbital measurements such as perihelion distance (q), aphelion distance (Q), and semi-major axis (a). NASA CNEOS defines Near-Earth Asteroid groups using these orbital values.

Orbit Class	Full Name	Meaning
`AMO`	Amor	Earth-approaching asteroids whose orbits are outside Earth’s orbit but still near Earth.
`APO`	Apollo	Earth-crossing asteroids with semi-major axes larger than Earth’s.

Orbit Class	Full Name	Meaning
`ATE`	Aten	Earth-crossing asteroids with semi-major axes smaller than Earth's.
`IEO`	Interior-Earth Object	Asteroids whose orbits are contained entirely inside Earth's orbit.
`MCA`	Mars-crossing Asteroid	An asteroid class for objects that cross Mars's orbit.

The Asteroid Dataset is split into 3 categories of attributes, identification (spkid), classifiers (NEO, PHA, orbit_class) and physical properties (H, diameter)

The NeoW dataset is split similarly into categories of attributes such as identification (Neo Reference ID), descriptors (Name, Magnitude) and Neo events (Approach date, velocity, miss distance).

1.3) Purpose

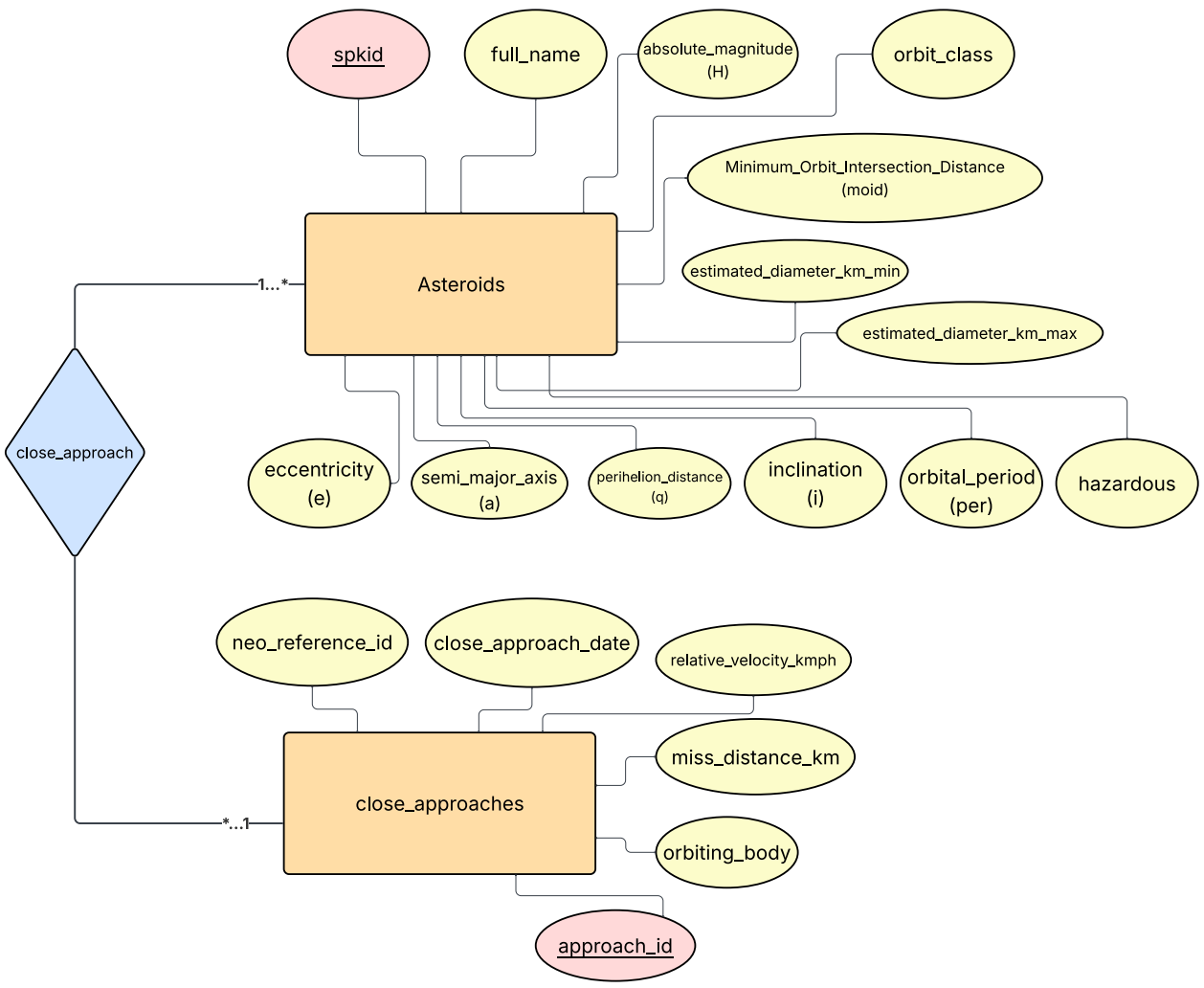
This dataset helps us answer questions about the quantity of NEOs and analyze their qualities.

2.1) Database Design

1NF: The schema satisfies First Normal Form because each relation stores atomic values only, with no repeating groups or multi-valued attributes. In the **asteroids_reduced** relation, each row represents a single asteroid identified by **spkid**. In the **close_approach_reduced** relation, each row represents a single close-approach event identified by **approach_id**. All attributes store single values such as one date, one velocity, or one miss distance.

2NF: Both relations satisfy Second Normal Form. Each relation is already in First Normal Form, and each uses a single-attribute primary key (**spkid** for **asteroids_reduced** and **approach_id** for **close_approach_reduced**). Because the primary keys are not composite, partial dependencies cannot occur. Therefore, every non-key attribute is fully functionally dependent on the whole primary key, and both relations are in 2NF.

3NF: Finally, both relations satisfy Third Normal Form because any attributes that were transitively dependent were removed from the base tables or relocated to the entity they actually describe. In **asteroids_reduced**, the primary key **spkid** determines the stored asteroid-level attributes directly. In **close_approach_reduced**, the primary key **approach_id** determines only event-level attributes, while asteroid-level attributes that would depend on **neo_reference_id** rather than **approach_id** were removed or moved to **asteroids_reduced**(such as asteroid dimensions). Therefore, no non-key attribute depends on another non-key attribute, and both relations are in 3NF.



Notably more or less attributes could have been chosen for more or less querying capabilities, however these attribute's that have been left out lack significant public documentation around their meaning. The reduction of attributes also helps the keep the scope

of this project more narrow.

The initial schema was created using SQL DDL statements before being reduced into the final normalized schema. Although the attributes will be reduced, this is a form of exercise that is important within database design. Many values are highly precise and as such require DOUBLE PRECISION. BOOLs and INTs are distributed to values that T/F and real numbers respectively. Finally TEXT is used in order to represent names of objects or fields that have an alphanumeric mix to them.

2.2) Preprocessing

Not only does this project narrow scope my minimizing the available attributes but also the individual rows. Since the strongest JOIN is between **spkid** and **neo_reference_id** the decision to remove all classified asteroids that did not have NEO entries and vice versa was made. This helps create stronger referential consistency in our queries, negating the possibility for an event or asteroid to not interact with the complementary table. This is not required for 3NF but helps the consistency of our queries. This was done outside of the NEON database in a quasi-preprocessing stage where DataFrames were loaded with every single row across all datasets and entries were compared using pandas methods to keep only those rows with key relations.

```
# SELECT * FROM asteroid a WHERE EXISTS (SELECT 1 FROM close_approach c WHERE c.neo_reference_id = a.s
pkid);
# SELECT * FROM close_approach c WHERE EXISTS (SELECT 1 FROM asteroid a WHERE a.spkid = c.neo_referenc
e_id);
if(not preview_user):
  asteroid_ids = asteroid_df["spkid"]
  close_ids = close_approach_df["neo_reference_id"]

  asteroid_df = asteroid_df[asteroid_df["spkid"].isin(close_ids)].copy()
  close_approach_df = close_approach_df[close_approach_df["neo_reference_id"].isin(asteroid_ids)].copy
()
```

During preprocessing, the raw dataset attributes were cleaned and standardized before being inserted into the database. Several column names from the original CSV files were shortened, renamed, or made more consistent so they could be used more clearly in the relational schema. For example, close-approach distance and velocity attributes were renamed into clearer database-style attribute names, and the asteroid classification attribute was stored as **orbit_class**.

The preprocessing stage also created a primary key for the close_approach table. Since the close-approach dataset did not already contain a simple unique identifier for each event, an approach_id attribute was added so that each close-approach record could be uniquely identified. The asteroid table used spkid as its primary key because it already uniquely identifies each asteroid.

```
  asteroid_df = asteroid_df.drop(axis = 1, columns=["name", "prefix", "diameter", "albedo", "diameter_
sigma"]) # Nothing in these cols
  close_approach_df = close_approach_df.drop(axis = 1, columns=["Unnamed: 0", "Name", "Est Dia in M(mi
n)", "Est Dia in M(max)", "Est Dia in Miles(min)", "Est Dia in Miles(max)",
  "Est Dia in Feet(min)", "Est Dia in Feet(max)", "Relative Velocity km per sec", "Miles per hour"]) #
Redundant in these cols
  close_approach_df.insert(0, "approach_id", range(1, len(close_approach_df) + 1))
```

2.3) Table Creation and Imputation of Relations

This project will be implemented using NEON database which uses PostgreSQL, the following table is created and subsequently reduced via psycopg2 and SQLAlchemy in Google Colab.

```
CREATE TABLE asteroid_reduced(
    id TEXT, spkid INT PRIMARY KEY, full_name TEXT,
    pdes INT, neo BOOL, pha BOOL, H FLOAT, orbit_id TEXT,
    epoch FLOAT, epoch_mjd INT, epoch_cal FLOAT, equinox TEXT,
    e DOUBLE PRECISION, a DOUBLE PRECISION, q DOUBLE PRECISION, i DOUBLE PRECISION, om DOUBLE PRECISIO
N, w DOUBLE PRECISION,
    ma DOUBLE PRECISION, ad DOUBLE PRECISION, n DOUBLE PRECISION, tp DOUBLE PRECISION, tp_cal DOUBLE P
RECISION,
```

```
    per DOUBLE PRECISION, per_y DOUBLE PRECISION, moid DOUBLE PRECISION, moid_ld DOUBLE PRECISION,
    sigma_e DOUBLE PRECISION, sigma_a DOUBLE PRECISION, sigma_q DOUBLE PRECISION, sigma_i DOUBLE PRECI
SION,
    sigma_om DOUBLE PRECISION, sigma_w DOUBLE PRECISION, sigma_ma DOUBLE PRECISION, sigma_ad DOUBLE PR
ECISION,
    sigma_n DOUBLE PRECISION, sigma_tp DOUBLE PRECISION, sigma_per DOUBLE PRECISION, class TEXT, rms D
OUBLE PRECISION
);

CREATE TABLE close_approach_reduced(
    approach_id INT PRIMARY KEY, a_date DATE, neo_reference_id INT,
    FOREIGN KEY (neo_reference_id) REFERENCES asteroid_reduced(spkid),
    absolute_magnitude DOUBLE PRECISION, estimated_diameter_km_min DOUBLE PRECISION,
        estimated_diameter_km_max DOUBLE PRECISION, close_approach_date DATE,
    epoch_date_close_approach BIGINT, relative_velocity_km_per_hr DOUBLE PRECISION,
    miss_dist_a DOUBLE PRECISION, miss_dist_l DOUBLE PRECISION, miss_dist_km DOUBLE PRECISION, miss_di
st_m DOUBLE PRECISION,
    orbiting_body TEXT, orbit_id INT, orbit_determination_date TIMESTAMP,
    orbit_uncertainty INT, minimum_orbit_intersection DOUBLE PRECISION,
    jupiter_tisserand_invariant DOUBLE PRECISION, epoch_osculation DOUBLE PRECISION,
    eccentricity DOUBLE PRECISION, semi_major_axis DOUBLE PRECISION, inclination DOUBLE PRECISION,
    asc_node_longitude DOUBLE PRECISION, orbital_period DOUBLE PRECISION,
    perihelion_distance DOUBLE PRECISION, perihelion_arg DOUBLE PRECISION,
    aphelion_dist DOUBLE PRECISION, perihelion_time DOUBLE PRECISION,
    mean_anomaly DOUBLE PRECISION, mean_motion DOUBLE PRECISION,
    equinox TEXT, hazardous BOOL
);
```

In order to get the schema to conform to 3NF we must drop or move the following attributes:

```
    ASTEROID_DELETE_LIST = [
        "name", "prefix", "diameter", "albedo", "diameter_sigma", "neo", "pha", "id", "pdes", "conditi
on_code",
        "epoch", "epoch_mjd", "epoch_cal", "equinox", "tp", "tp_cal", "per_y", "moid_ld", "moid_jup",
        "sigma_e", "sigma_a", "sigma_q", "sigma_i", "sigma_om", "sigma_w", "sigma_ma", "sigma_ad", "si
gma_n",
        "sigma_tp", "sigma_per", "rms", "w", "om", "ma", "n", "ad"
    ]

    CLOSE_APPROACH_DELETE_LIST = [
        "absolute_magnitude", "epoch_date_close_approach",
        "miss_dist_a", "miss_dist_l", "miss_dist_m", "orbit_id", "orbit_determination_date", "orbit_un
certainty",
        "minimum_orbit_intersection", "jupiter_tisserand_invariant", "epoch_osculation", "eccentricit
y",
        "semi_major_axis", "inclination", "asc_node_longitude", "orbital_period", "perihelion_distanc
e",
        "perihelion_arg", "aphelion_dist", "perihelion_time", "mean_anomaly", "mean_motion", "equinox"
    ]
```

```
ALTER TABLE asteroid_reduced DROP COLUMN IF EXISTS name;
ALTER TABLE asteroid_reduced DROP COLUMN IF EXISTS prefix;
ALTER TABLE asteroid_reduced DROP COLUMN IF EXISTS diameter;
...
ALTER TABLE close_approach_reduced DROP COLUMN IF EXISTS absolute_magnitude;
ALTER TABLE close_approach_reduced DROP COLUMN IF EXISTS miss_dist_a;
```

```
ALTER TABLE close_approach_reduced DROP COLUMN IF EXISTS inclination;
...
```

We keep the attribute for estimated_diameter_km_min/max in close_approach reduced so we can transfer it over before we drop it. Before we do that we need to create the columns for these attributes

```
-- Add Column
ALTER TABLE asteroid_reduced
ADD COLUMN
  estimated_diameter_km_min DOUBLE PRECISION PRECISION,
ADD COLUMN
  estimated_diameter_km_max DOUBLE PRECISION PRECISION;

-- Set New Columns
UPDATE asteroid_reduced a SET
  estimated_diameter_km_min = c.estimated_diameter_km_min,
  estimated_diameter_km_max = c.estimated_diameter_km_max
FROM close_approach_reduced c
WHERE a.spkid = c.neo_reference_id;

-- Delete Columns
ALTER TABLE close_approach_reduced
DROP COLUMN
  estimated_diameter_km_min,
DROP COLUMN
  estimated_diameter_km_max
```

3.1) Querying the Database

1. Which asteroids have the most recorded close approaches?

```
SELECT
    a.spkid,
    a.full_name,
    COUNT(c.approach_id) AS close_approach_count
FROM asteroid_reduced AS a
JOIN close_approach_reduced AS c
ON a.spkid = c.neo_reference_id
GROUP BY
    a.spkid,
    a.full_name
ORDER BY close_approach_count DESC limit 5;
```

- This query shows which asteroids have the most recorded close-approach events, with 277810 (2006 FV35) having the highest count at 251 records.
- Because 2006 FV35 appears so frequently in the close-approach records, it may be a well-tracked object in this dataset, making it useful for repeated-event analysis.”

2. Which asteroid have come closest to Earth based on minimum miss_dist_km?

```
SELECT
    a.spkid,
    a.full_name,
```

```
    MIN(c.miss_dist_km) AS closest_recorded_miss
FROM asteroid_reduced AS a
JOIN close_approach_reduced AS c
ON a.spkid = c.neo_reference_id
GROUP BY
    a.spkid,
    a.full_name
ORDER BY closest_recorded_miss ASC
LIMIT 5;
```

- This query shows the closest recorded miss distance for each asteroid, with (2011 CQ1) having the smallest value at about 11,851.67 km.
- This is useful because it identifies the asteroids with the most extreme close passes in the dataset.

3. What are the top 5 closest approaches to Earth in the dataset?

```
SELECT
    a.spkid,
    a.full_name,
    c.miss_dist_km,
    c.orbiting_body
  FROM asteroid_reduced as a
  JOIN close_approach_reduced AS c
  ON a.spkid = c.neo_reference_id
  ORDER BY miss_dist_km ASC LIMIT 5
```

- This query shows the five closest individual close-approach events, with the closest event also belonging to (2011 CQ1).
- This is different from the asteroid-level query because it focuses on specific close-approach records rather than one summary per asteroid.

4. Which orbit classes have the greatest number of asteroids?

```
SELECT
    orbit_class,
    COUNT(*) AS asteroid_count
FROM asteroid_reduced
GROUP BY orbit_class
ORDER BY asteroid_count DESC
LIMIT 5;
```

- This query shows which orbit classes contain the most asteroids in the reduced dataset by grouping asteroid records by orbit_class.
- The results are useful because they show which orbital categories are most common among the NEOs retained in the final database.

5. Which orbit classes have the smallest average MOID (Minimum Orbit Intersection Distance)?

```
SELECT
    orbit_class,
    COUNT(moid) AS asteroid_count,
    AVG(moid) AS average_moid
  FROM asteroid_reduced
  GROUP BY
```

```
    orbit_class
  ORDER BY average_moid ASC LIMIT 5
```

- This query shows which orbit classes have the lowest average minimum orbit intersection distance, with ATE and APO having the smallest averages.
- This suggests that ATE and APO asteroids tend to have orbits that come closer to Earth's orbit than the other listed classes.

6. How many close approaches occur in each year?

```
SELECT
    DATE_TRUNC('year', a_date) AS year,
    COUNT(*) AS total_approaches
  FROM close_approach_reduced
  GROUP BY
    year
  ORDER BY total_approaches DESC LIMIT 5
```

- This query shows which years have the highest number of recorded close approaches, with 2019 having the most at 5,737 records.
- This is useful for identifying years with especially high close-approach activity in the dataset.

7. Which orbit classes have the highest average close-approach velocity?

```
SELECT
    a.orbit_class,
    AVG(c.relative_velocity_km_per_hr) AS velocity
  FROM asteroid_reduced as a
  JOIN close_approach_reduced AS c
  ON a.spkid = c.neo_reference_id
  GROUP BY
    a.orbit_class
  ORDER BY velocity DESC LIMIT 5
```

- This query shows the average relative velocity for close approaches grouped by orbit class, with IEO having the highest average velocity.
- This suggests that some orbit classes tend to pass Earth at higher speeds than others.

8. Which orbit classes have the largest average estimated asteroid diameter?

```
SELECT
    orbit_class,
    AVG((estimated_diameter_km_min + estimated_diameter_km_max) / 2) AS avg_diameter
  FROM asteroid_reduced
  GROUP BY
    orbit_class
  ORDER BY avg_diameter DESC LIMIT 5
```

- This query shows the average estimated diameter midpoint for each orbit class, with IEO having the largest average diameter.
- This is useful because it compares physical asteroid size across different orbital groups.

9. Among hazardous asteroids, which ones have the smallest miss distances?

```sql
SELECT
    a.spkid,
    a.full_name,
    c.miss_dist_km,
    c.hazardous
  FROM asteroid_reduced as a
  JOIN close_approach_reduced AS c
  ON a.spkid = c.neo_reference_id
  WHERE c.hazardous IS true
  ORDER BY c.miss_dist_km ASC LIMIT 5
```

- This query shows hazardous asteroid close approaches with the smallest miss distances, with (2019 CD2) having the closest hazardous approach.
- This helps identify hazardous records that may be more important to monitor because they combine hazardous status with close distance.

10. Do hazardous asteroids have a higher average relative velocity than non-hazardous ones?

```sql
SELECT
    c.hazardous,
    AVG(c.relative_velocity_km_per_hr) AS velocity
  FROM asteroid_reduced as a
  JOIN close_approach_reduced AS c
  ON a.spkid = c.neo_reference_id
  GROUP BY
    hazardous
  ORDER BY velocity DESC
```

- This query compares average relative velocity between hazardous and non-hazardous records, showing that hazardous records have the higher average velocity.
- This is interesting because hazardous close approaches in this dataset are not only closer-risk records but also tend to move faster on average.

11. Which hazardous close approaches are closer than the average miss distance?

```sql
SELECT
  approach_id,
  neo_reference_id,
  close_approach_date,
  miss_dist_km,
  hazardous
FROM close_approach_reduced
WHERE hazardous IS TRUE
AND miss_dist_km <
  (SELECT AVG(miss_dist_km)
  FROM close_approach_reduced
  WHERE miss_dist_km IS NOT NULL)
ORDER BY miss_dist_km ASC
LIMIT 5;
```

- This query shows hazardous close-approach records whose miss distances are below the dataset's average miss distance.
- This is useful because it filters hazardous records down to those that are unusually close compared with the overall dataset.

12. Which orbit classes have an average MOID lower than the overall average MOID?

```sql
SELECT
  orbit_class,
  COUNT(moid) AS asteroid_count,
  AVG(moid) AS average_moid
FROM asteroid_reduced
WHERE moid IS NOT NULL
GROUP BY orbit_class
HAVING AVG(moid) <
  (SELECT AVG(moid)
  FROM asteroid_reduced
  WHERE moid IS NOT NULL)
ORDER BY average_moid ASC;
```

- This query shows which orbit classes have average MOID values below the dataset-wide average, returning only ATE and APO.
- This supports the finding that ATE and APO are the orbit classes most closely associated with lower Earth-orbit intersection distances.

13. How can we create a reusable combined table of asteroid details and close-approach events for later analysis?

```sql
CREATE OR REPLACE VIEW asteroid_close_approach_view AS
SELECT
    a.spkid,
    a.full_name,
    a.h AS absolute_magnitude,
    a.orbit_class,
    a.moid,
    a.estimated_diameter_km_min,
    a.estimated_diameter_km_max,
    c.approach_id,
    c.close_approach_date,
    c.relative_velocity_km_per_hr,
    c.miss_dist_km,
    c.orbiting_body,
    c.hazardous
FROM asteroid_reduced AS a
JOIN close_approach_reduced AS c
    ON a.spkid = c.neo_reference_id;

SELECT
  *
FROM asteroid_close_approach_view
LIMIT 5;
```

- This view combines asteroid attributes with close-approach event attributes into one reusable joined structure.
- This is useful because it simplifies later queries by avoiding repeated joins between asteroid_reduced and close_approach_reduced.

4. Conclusion

This project used two related asteroid datasets to design a relational database for analyzing near-Earth object records and close-approach events. The final schema separates asteroid-level from close-approach events, which supports normalization through Third Normal Form and allows meaningful joins between asteroid records and approach records.

The SQL analysis showed that some asteroids appear many times in close-approach records, certain orbit classes such as ATE and APO tend to have lower average MOID values, and hazardous close approaches tend to have higher average relative velocities than

non-hazardous ones. Overall, the database demonstrates how structured relational data can be used to identify patterns in asteroid size, orbit class, velocity, hazardous status, and close-approach distance.